1

VARIANTES DE ACTOR-CRITIC

VARIANTES

- ✓ A2C (2016)
- ✓ A3C (2016)
- ✓ DDPG (2016)
- ✓ SAC (2018)
- ✓ TRPO (2015)
- ✓ PPO (2017)
- ✓ TD3 (2018)
- ✓ IMPALA (2018)

ASYNCHRONOUS ADVANTAGE ACTOR-CRITIC (A3C)

- Este método publicado con el título "Métodos asincrónicos para aprendizaje por refuerzo profundo" (Mnih et al., 2016).

Asynchronous Methods for Deep Reinforcement Learning

Volodymyr Mnih[1] VMNIH@GOOGLE.COM
Adrià Puigdomènech Badia[1] ADRIAP@GOOGLE.COM
Mehdi Mirza[1,2] MIRZAMOM@IRO.UMONTREAL.CA
Alex Graves[1] GRAVESA@GOOGLE.COM
Tim Harley[1] THARLEY@GOOGLE.COM
Timothy P. Lillicrap[1] COUNTZERO@GOOGLE.COM
David Silver[1] DAVIDSILVER@GOOGLE.COM
Koray Kavukcuoglu [1] KORAYK@GOOGLE.COM

[1] Google DeepMind
[2] Montreal Institute for Learning Algorithms (MILA), University of Montreal

Abstract

We propose a conceptually simple and lightweight framework for deep reinforcement learning that uses asynchronous gradient descent for optimization of deep neural network controllers. We present asynchronous variants of four standard reinforcement learning algorithms and show that parallel actor-learners have a stabilizing effect on training allowing all four methods to successfully train neural network controllers. The best performing method, an asynchronous variant of actor-critic, surpasses the current state-of-the-art on the Atari domain while training for half the time on a single multi-core CPU instead of a GPU. Furthermore, we show that asynchronous actor-critic succeeds on a wide variety of continuous motor control problems as well as on a new task of navigating random 3D mazes using a visual input.

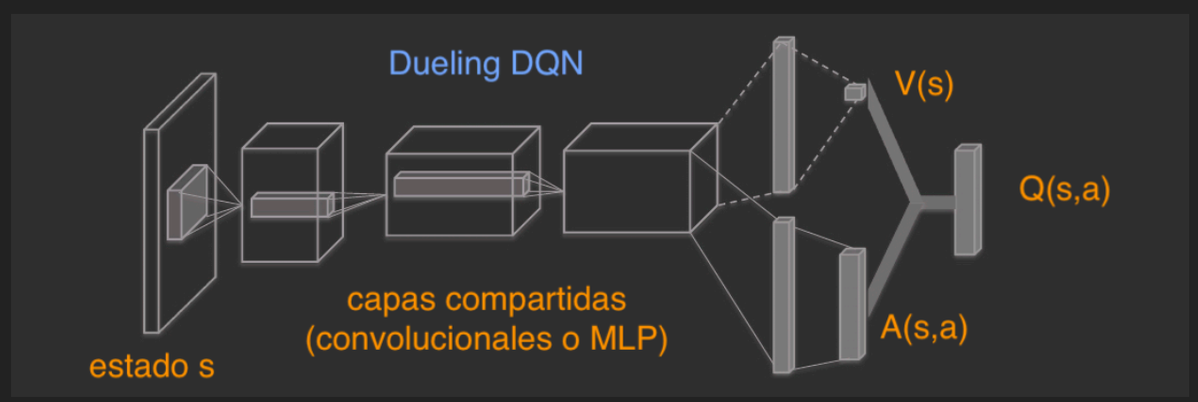
line RL updates are strongly correlated. By storing the agent's data in an experience replay memory, the data can be batched (Riedmiller, 2005; Schulman et al., 2015a) or randomly sampled (Mnih et al., 2013; 2015; Van Hasselt et al., 2015) from different time-steps. Aggregating over memory in this way reduces non-stationarity and decorrelates updates, but at the same time limits the methods to off-policy reinforcement learning algorithms.

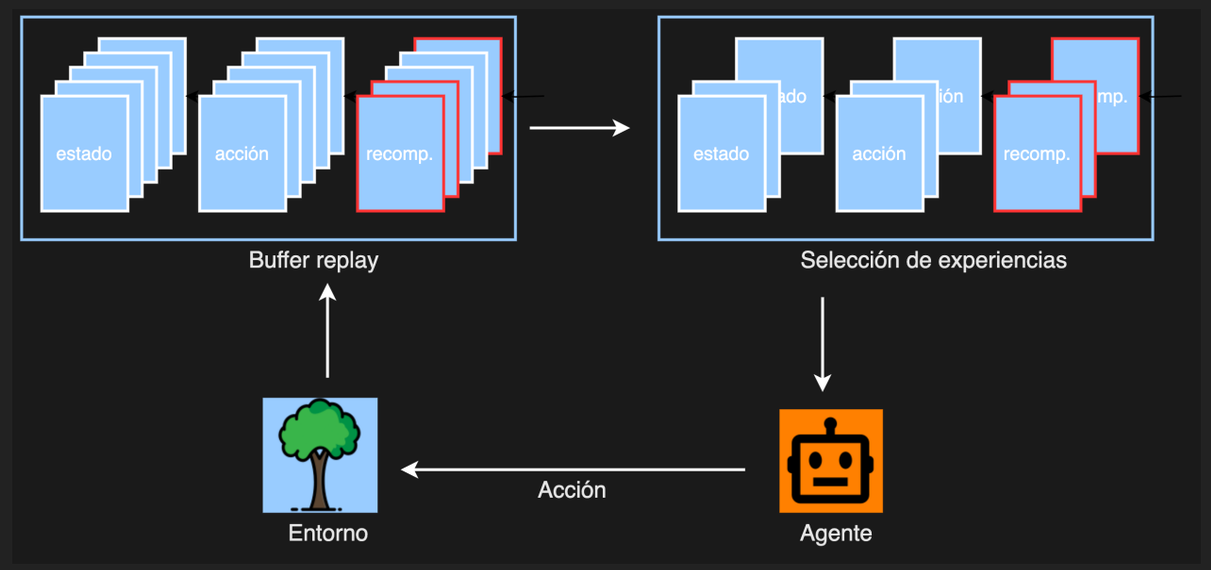
Deep RL algorithms based on experience replay have achieved unprecedented success in challenging domains such as Atari 2600. However, experience replay has several drawbacks: it uses more memory and computation per real interaction; and it requires off-policy learning algorithms that can update from data generated by an older policy.

In this paper we provide a very different paradigm for deep reinforcement learning. Instead of experience replay, we asynchronously execute multiple agents in parallel, on multiple instances of the environment. This parallelism also decorrelates the agents' data into a more stationary process,

COMO FUNCIONA A3C?

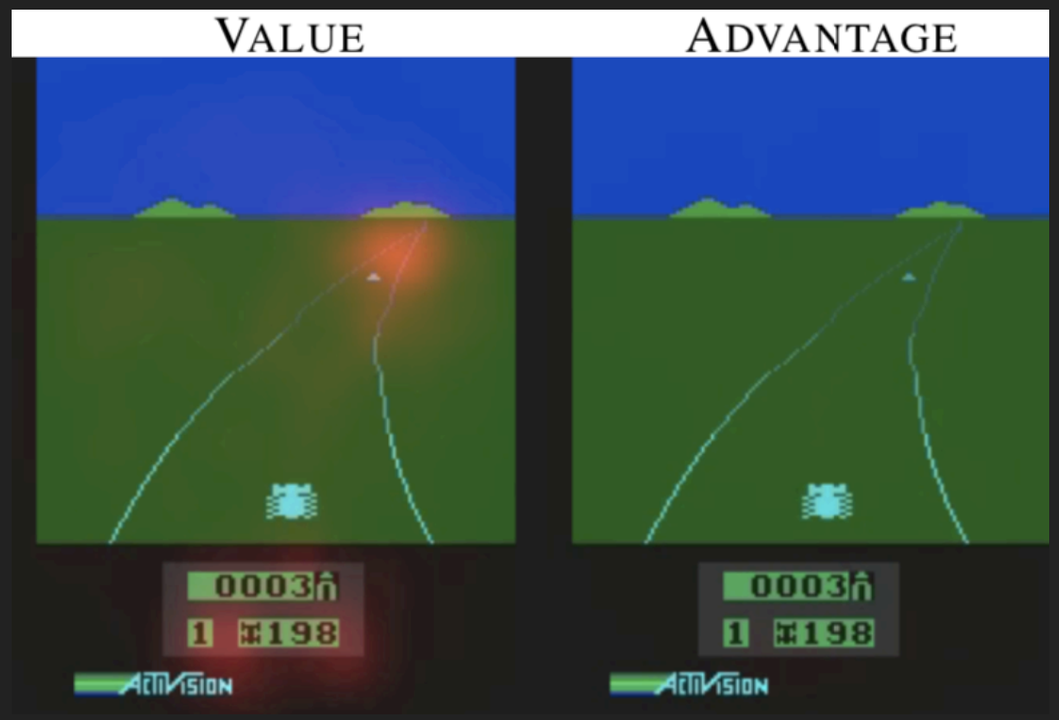
- A3C se basa (parcialmente) en la variante Dueling DQN:
  - ✓ Usa la función de ventaja A(s, a).
  - ✓ Reemplaza Replay Buffer por una arquitectura de varios actores en lugar de uno.

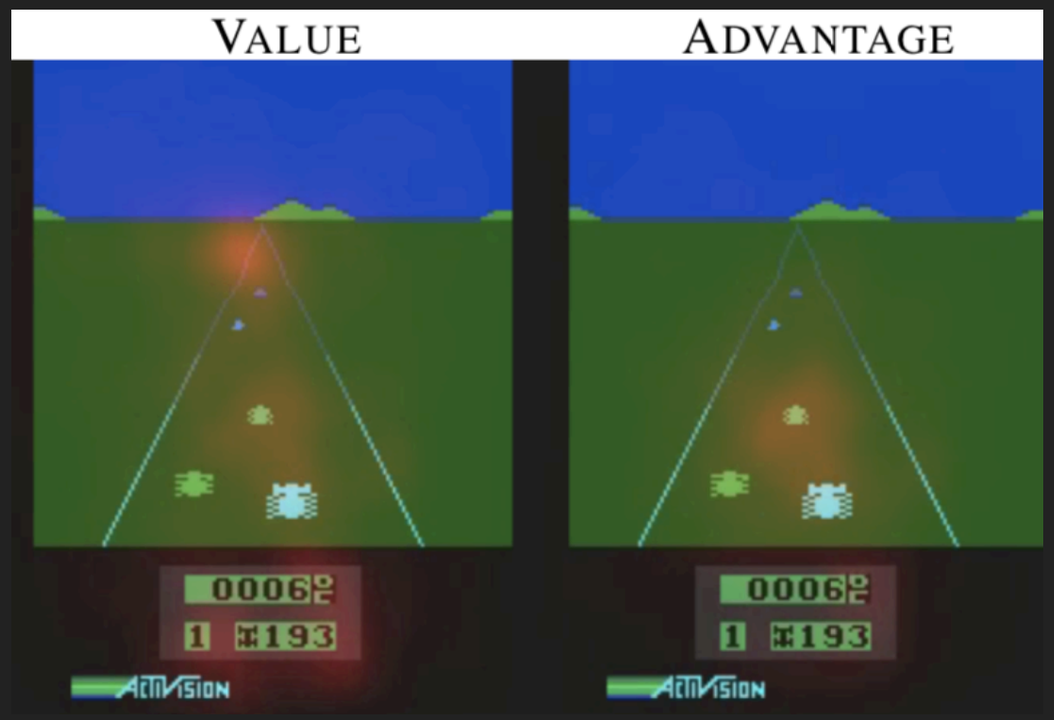




FUNCIÓN DE VENTAJA A(S, A)

- En Dueling DQN la solución consistía en desacoplar la función de valor Q en dos términos que respondían si:
  - ✓ ¿cuál es el valor de estar en el estado s?
  - ✓ ¿cuál es la ventaja de tomar una acción a estando en ese estado s?



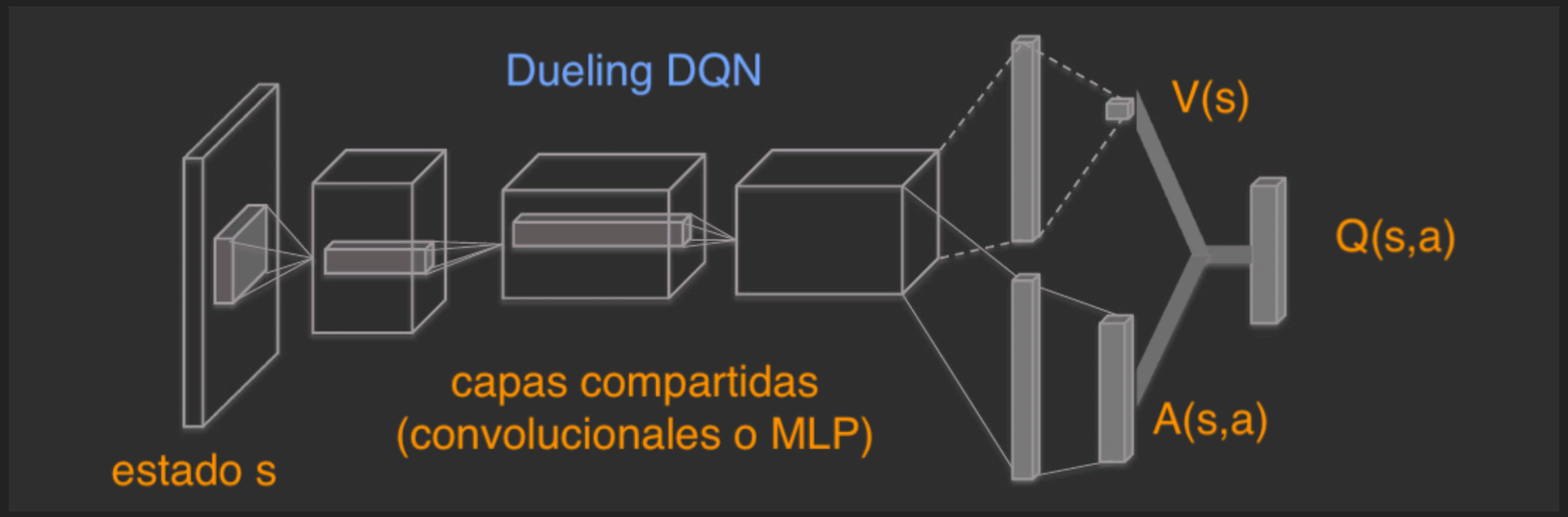


FUNCIÓN DE VENTAJA A(S, A)

- El desacople de Q(s, a) se reflejaba matemáticamente como:

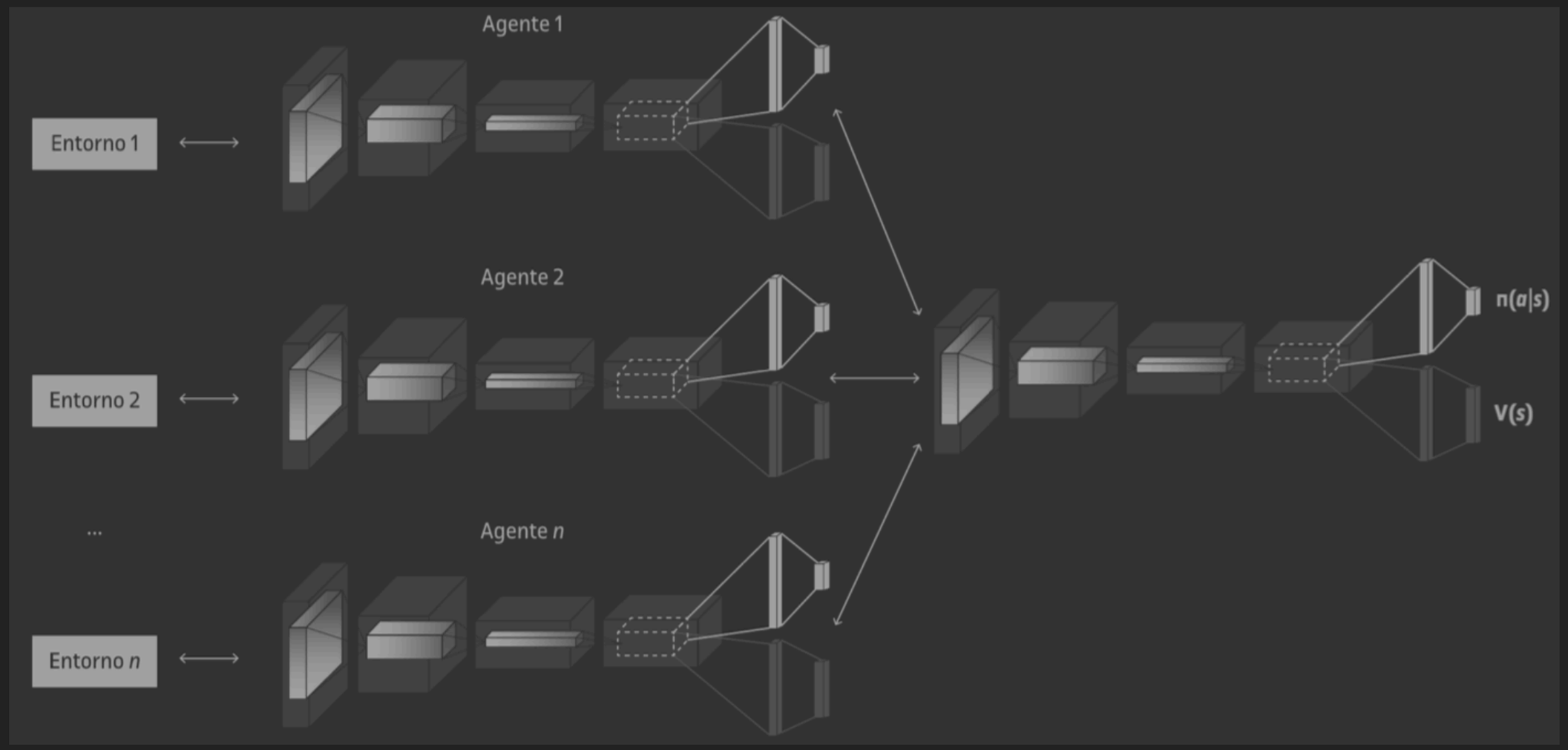
$$Q(s, a) = A(s, a) + V(s)$$

- y también se reflejaba en la arquitectura de la red neuronal.
- A3C toma esta arquitectura de Dueling DQN para su implementación.



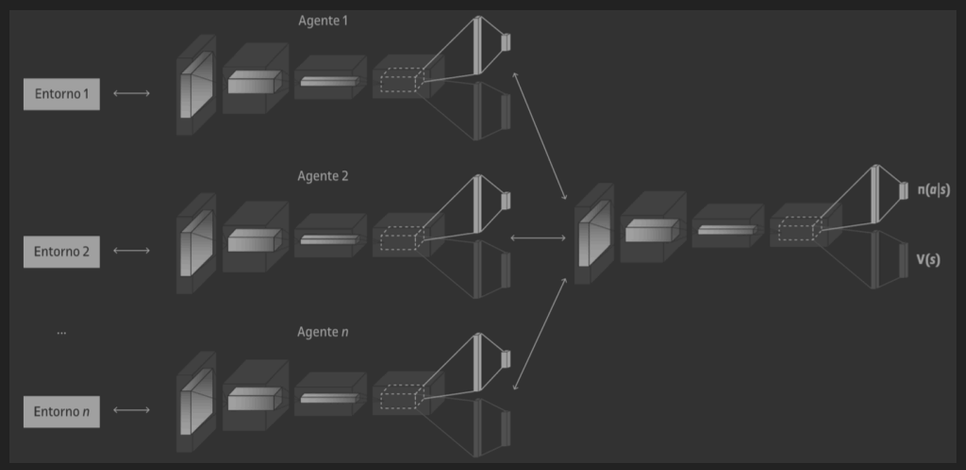
ARQUITECTURA DE VARIOS ACTORES

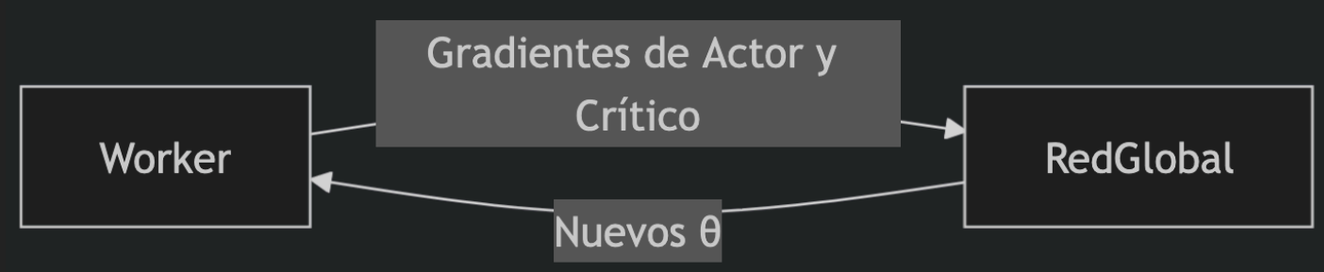
- Dado que el Replay Buffer consume mucha memoria y cómputo A3C propone usar múltiples agentes (actor-learners o workers) ejecutándose en paralelo de forma asíncrona en diferentes hilos (threads) de una CPU estándar.



ARQUITECTURA DE VARIOS ACTORES DE A3C

- Los agentes en forma periódica, asíncrona e independiente, actualizan la función de valor global (global network).
- Después de que se actualiza la global networking, los agentes actualizan sus propios gradientes con los de la red global y continúan normalmente su proceso de aprendizaje n-step hasta la siguiente actualización.
- Toda vez que un agente se actualiza está transfiriendo su información a la global networking.





ARQUITECTURA DE VARIOS ACTORES DE A3C

- El Advantage A(s,a) es un cálculo temporal que ocurre en cada worker, derivado de sus propias experiencias y su copia local del Crítico.

Worker (Local):

|--> Genera (s, a, r, s')

|--> Calcula V(s) con Crítico local

|--> Calcula Recompensa_descontada = $\Sigma\gamma r$

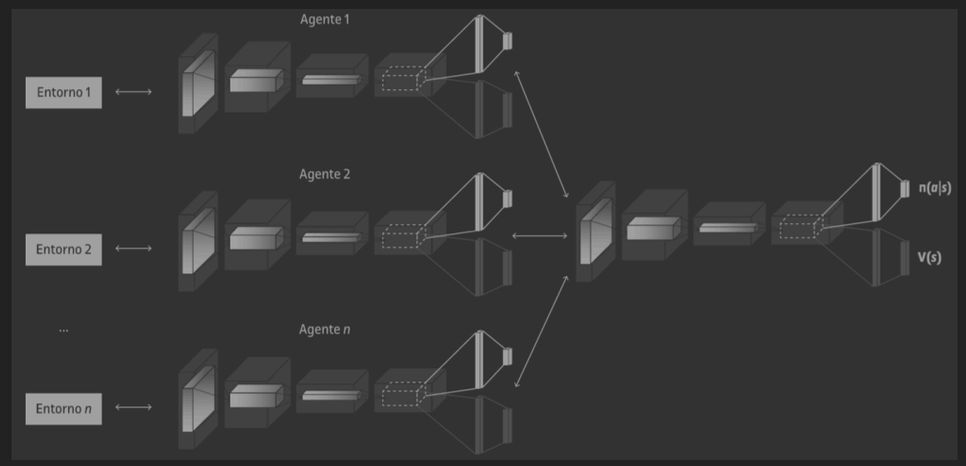
|--> Calcula A(s,a) = Recompensa_descontada - V(s)

|--> $\nabla\theta$_actor = $-\nabla\log\pi(a|s) * A(s,a)$

|--> $\nabla\theta$_critic = $\nabla(A(s,a)^2$

|--> Envía $\nabla\theta$_actor + $\nabla\theta$_critic a la red global

(envía gradientes a la red global)

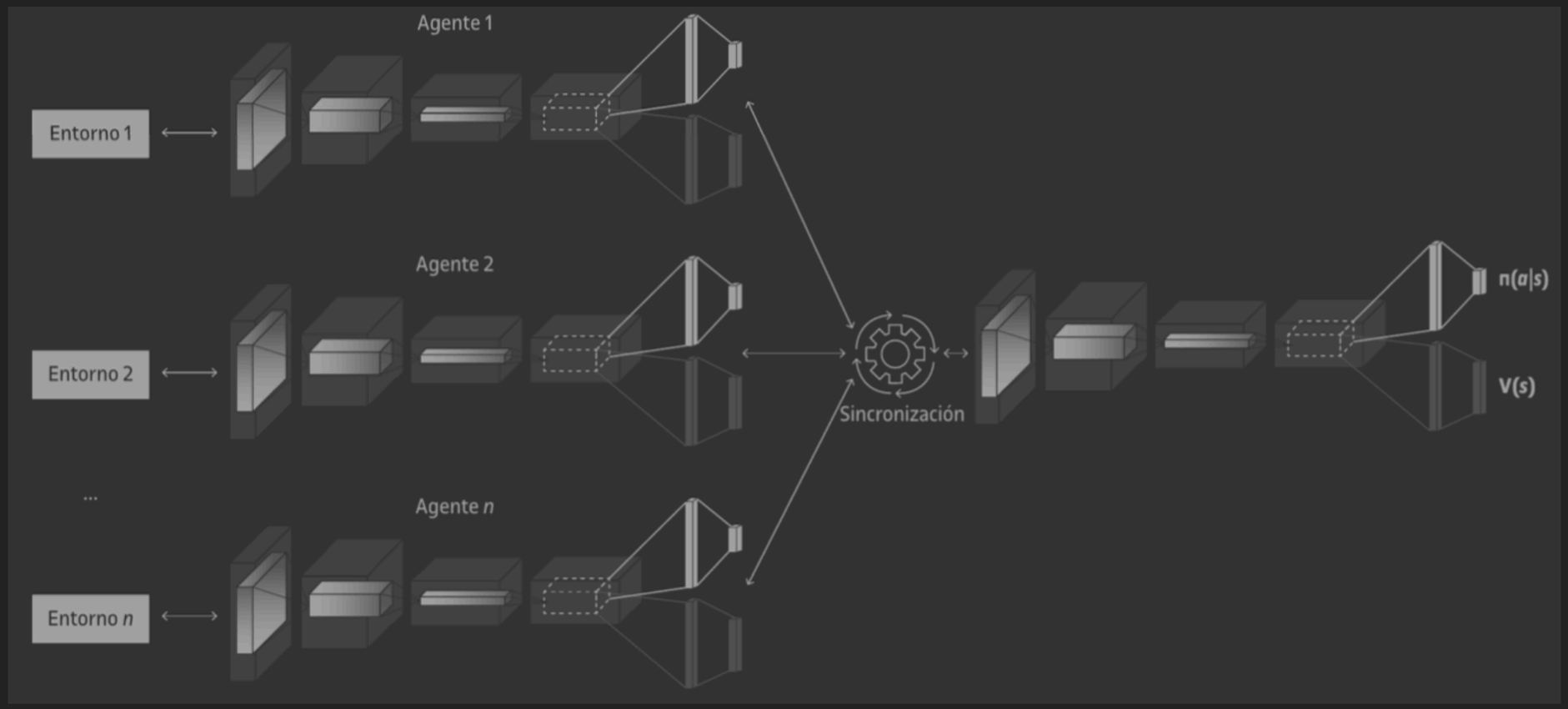


PROBLEMAS DE A3C

- A3C tiene problemas con el overlapping de gradientes (por su cualidad asíncrona).
- A3C requiere ajustes complejos para funcionar bien en entornos modernos.
- A3C fue parte de la biblioteca RLlib hasta la aparición de la versión Ray 2.0 en 2023 (la versión actual de Ray es 2.49.0, Agosto 2025).
- A3C es una variante de Actor-Critic obsoleta y en creciente desuso, no así A2C.
- A3C es menos eficiente y menos estable que A2C.

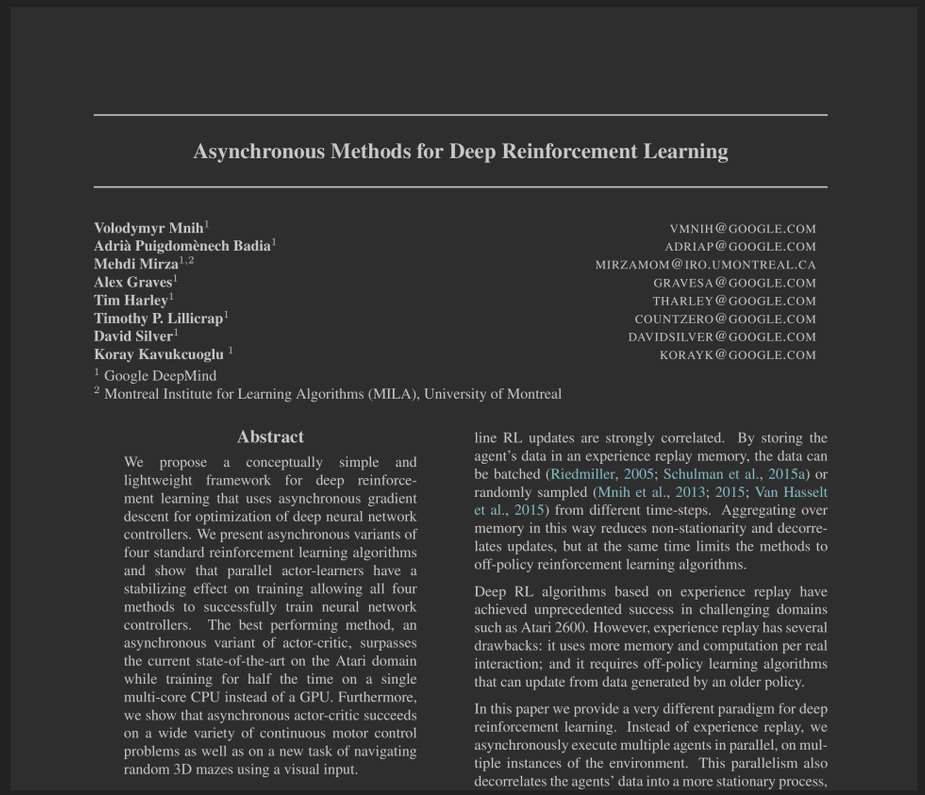
ADVANTAGE ACTOR-CRITIC (A2C)

- A2C es el equivalente síncrono de A3C.
- A2C también tiene varios agentes que se entrenan a la vez, pero luego de que todos finalizan su proceso de aprendizaje (luego de n pasos) se sincronizan todos sus parámetros y la global networking se actualiza.



CONTRIBUCIONES DE A2C

- Reducción de la varianza en las actualizaciones del actor => acelera la convergencia.
- Combina paralelización sincrónica + estimación de ventaja para lograr un aprendizaje más estable y eficiente que los métodos clásicos.



Asynchronous Methods for Deep Reinforcement Learning

Volodymyr Mnih[1] VMNIH@GOOGLE.COM
Adrià Puigdomènech Badia[1] ADRIAP@GOOGLE.COM
Mehdi Mirza[1,2] MIRZAMOM@IRO.UMONTREAL.CA
Alex Graves[1] GRAVESA@GOOGLE.COM
Tim Harley[1] THARLEY@GOOGLE.COM
Timothy P. Lillicrap[1] COUNTZERO@GOOGLE.COM
David Silver[1] DAVIDSILVER@GOOGLE.COM
Koray Kavukcuoglu [1] KORAYK@GOOGLE.COM
[1] Google DeepMind
[2] Montreal Institute for Learning Algorithms (MILA), University of Montreal

Abstract

We propose a conceptually simple and lightweight framework for deep reinforcement learning that uses asynchronous gradient descent for optimization of deep neural network controllers. We present asynchronous variants of four standard reinforcement learning algorithms and show that parallel actor-learners have a stabilizing effect on training allowing all four methods to successfully train neural network controllers. The best performing method, an asynchronous variant of actor-critic, surpasses the current state-of-the-art on the Atari domain while training for half the time on a single multi-core CPU instead of a GPU. Furthermore, we show that asynchronous actor-critic succeeds on a wide variety of continuous motor control problems as well as on a new task of navigating random 3D mazes using a visual input.

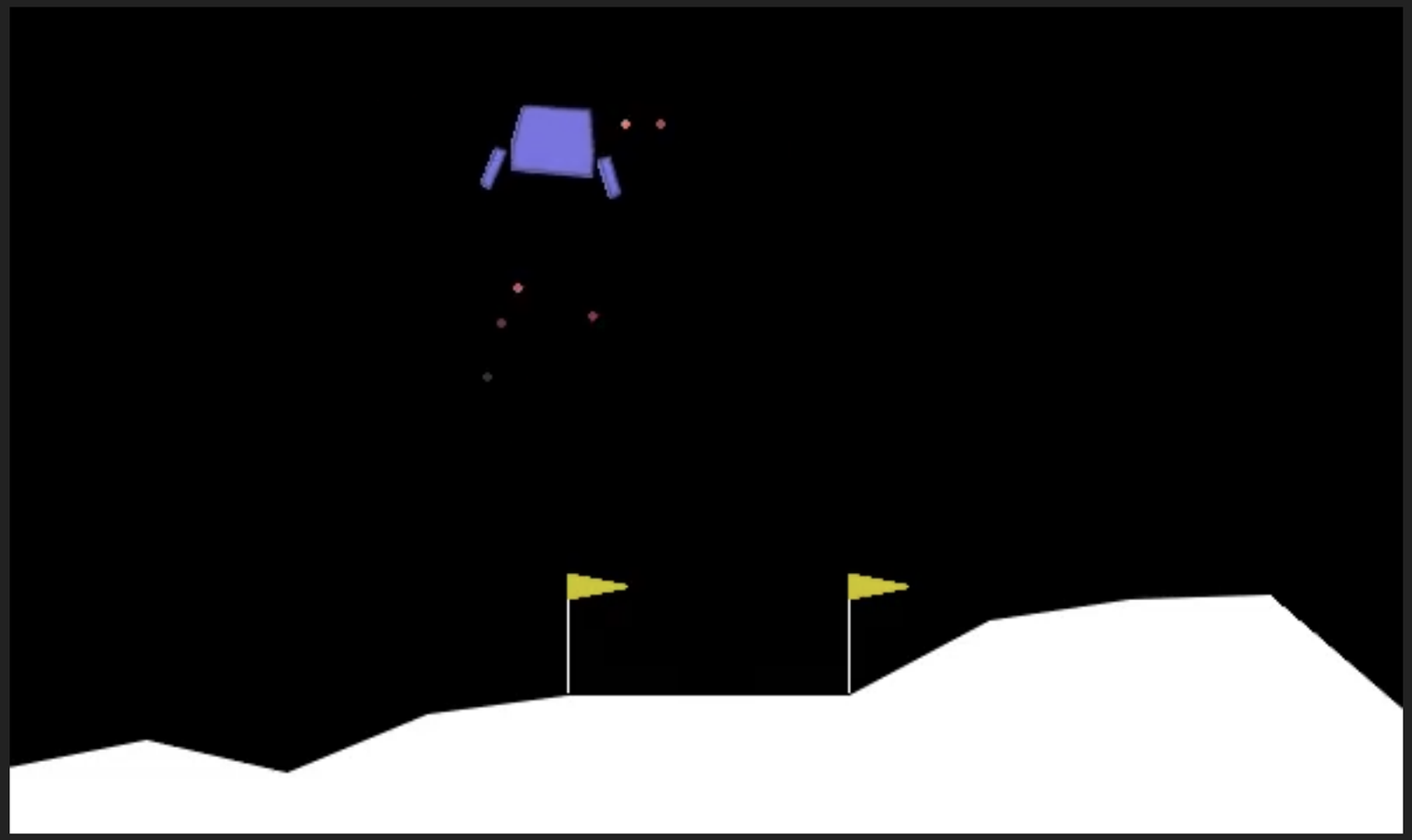
line RL updates are strongly correlated. By storing the agent's data in an experience replay memory, the data can be batched (Riedmiller, 2005; Schulman et al., 2015a) or randomly sampled (Mnih et al., 2013; 2015; Van Hasselt et al., 2015) from different time-steps. Aggregating over memory in this way reduces non-stationarity and decorrelates updates, but at the same time limits the methods to off-policy reinforcement learning algorithms.

Deep RL algorithms based on experience replay have achieved unprecedented success in challenging domains such as Atari 2600. However, experience replay has several drawbacks: it uses more memory and computation per real interaction; and it requires off-policy learning algorithms that can update from data generated by an older policy.

In this paper we provide a very different paradigm for deep reinforcement learning. Instead of experience replay, we asynchronously execute multiple agents in parallel, on multiple instances of the environment. This parallelism also decorrelates the agents' data into a more stationary process,

EJEMPLO

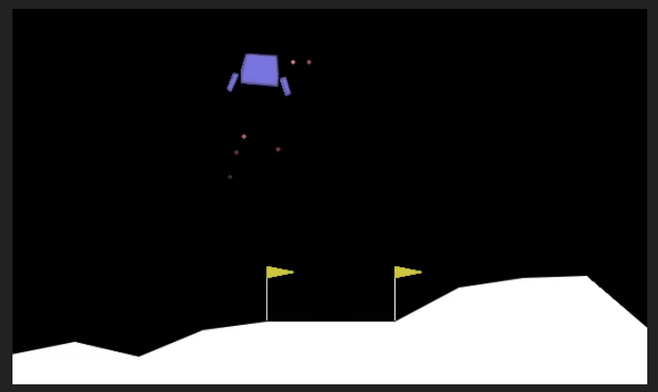
- Aprendizaje del alunizaje con LunaLander de Gymnasium y con la biblioteca de agentes Stable Baseline 3.



PARÁMETROS EN SB3

- policy='MlpPolicy': Especifica la arquitectura de las redes neuronales para el Actor y el Crítico. 'MlpPolicy' significa que se usarán Redes Neuronales Perceptrón Multicapa (Multi-Layer Perceptrons).
- env=train_env: El entorno de entrenamiento con el que interactuará el agente (LunarLander).
- n_steps=5: El número de pasos que cada "worker" (o el único worker en A2C) ejecuta en el entorno para recolectar experiencias antes de calcular las actualizaciones de los modelos. Se recolectan 5 transiciones (s, a, r, s') antes de hacer un paso de optimización.
- gamma=0.99: El factor de descuento (γ).

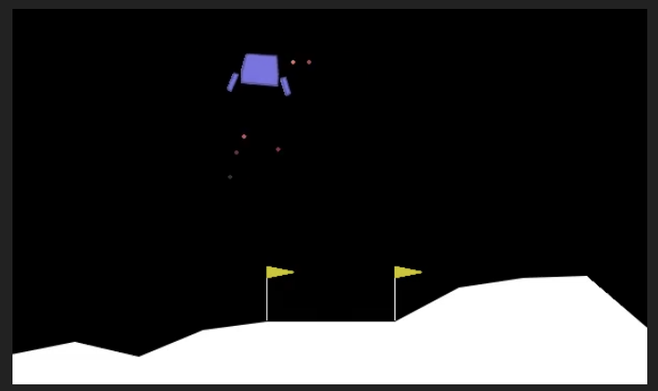
```
# creo el agente A2C
model = A2C(
    policy='MlpPolicy',
    env=train_env,
    n_steps=5,
    gamma=0.99,
    gae_lambda=1.0,
    ent_coef=0.0,
    vf_coef=0.5,
    max_grad_norm=0.5,
    learning_rate=7e-4,
    use_rms_prop=True,
    verbose=1,
    seed=42
)
```



PARÁMETROS EN SB3

- gae_lambda=1.0: Parámetro Lambda para Generalized Advantage Estimation (GAE). GAE es una técnica para estimar la función de ventaja (A(s, a) = Q(s, a) - V(s)) que balancea sesgo y varianza. lambda=1.0 significa que la estimación de la ventaja se parecerá mucho a la ventaja calculada con Monte Carlo (usando retornos completos dentro del lote de n_steps), lo que generalmente reduce el sesgo pero puede aumentar la varianza. lambda=0 se parecería más a la ventaja TD(0).
- ent_coef=0.0: Coeficiente de entropía (c_ent). Es el peso que se le da al término de regularización por entropía en la función de pérdida del Actor. La regularización por entropía anima a la política a ser más estocástica (exploratoria). Un valor de 0.0 significa que no se está usando explícitamente esta regularización (lo cual es un poco inusual para A2C, a menudo se usa un valor pequeño como 0.01).

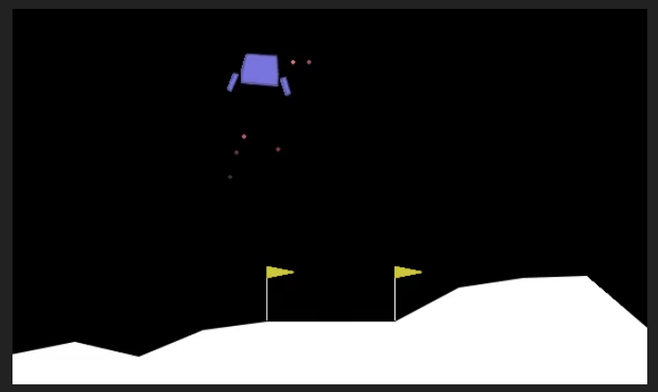
```
# creo el agente A2C
model = A2C(
    policy='MlpPolicy',
    env=train_env,
    n_steps=5,
    gamma=0.99,
    gae_lambda=1.0,
    ent_coef=0.0,
    vf_coef=0.5,
    max_grad_norm=0.5,
    learning_rate=7e-4,
    use_rms_prop=True,
    verbose=1,
    seed=42
)
```



PARÁMETROS EN SB3

- vf_coef=0.5: Coeficiente de la función de valor (c_vf). Es el peso que se le da a la pérdida del Crítico (value loss) en la función de pérdida total. Ayuda a balancear el aprendizaje de la política (Actor) y la función de valor (Crítico). Si vf_coef es muy alto, el Crítico dominará y el Actor aprenderá lentamente. Si es muy bajo, el Crítico no guiará bien al Actor (política inestable).
- max_grad_norm=0.5: Norma máxima del gradiente. Se utiliza para el recorte de gradientes (gradient clipping). Si la norma L2 total de los gradientes excede este valor, los gradientes se reescalan para que su norma sea igual a max_grad_norm. Esto previene que gradientes muy grandes desestabilicen el aprendizaje. Si los gradientes son [0.3, 0.4], su norma L2 es raiz(0.3^2+0.4^2)=0.50.32+0.42=0.5. Si max_grad_norm=0.5, estos gradientes no se recortan (ya que 0.5≤0.5≤0.5). Si fueran [0.6, 0.8] (norma 1.0), se reescalarían a [0.3, 0.4].

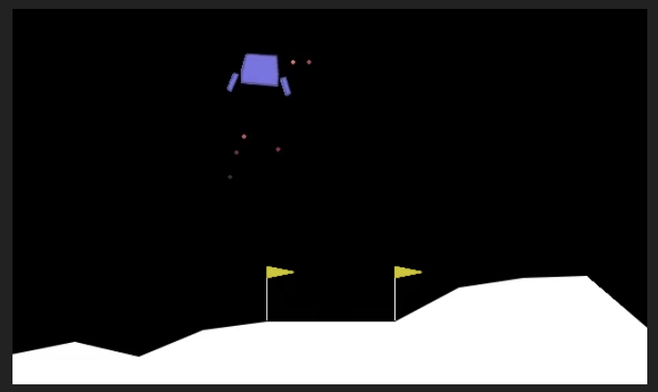
```python
# creo el agente A2C
model = A2C(
    policy='MlpPolicy',
    env=train_env,
    n_steps=5,
    gamma=0.99,
    gae_lambda=1.0,
    ent_coef=0.0,
    vf_coef=0.5,
    max_grad_norm=0.5,
    learning_rate=7e-4,
    use_rms_prop=True,
    verbose=1,
    seed=42
)
```



PARÁMETROS EN SB3

- learning_rate=7e-4: Tasa de aprendizaje (α o lr). Controla el tamaño de los pasos que da el optimizador (e.g., Adam, RMSProp) al actualizar los pesos de las redes neuronales. 7e-4 (o 0.0007) es un valor común.
- use_rms_prop=True: Indica si se debe usar el optimizador RMSProp. Si es False, Stable Baselines 3 suele usar ell optimizador adaptativo Adam por defecto.

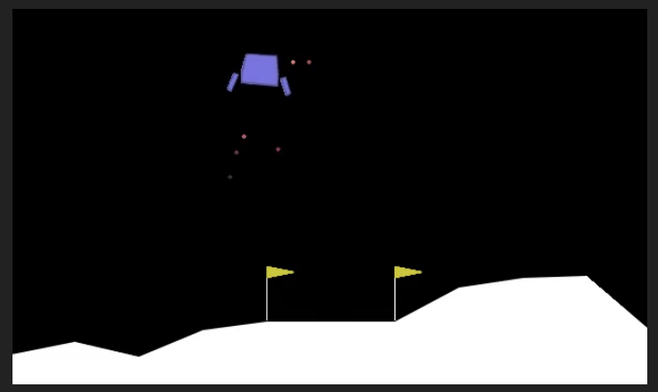
```
# creo el agente A2C
model = A2C(
    policy='MlpPolicy',
    env=train_env,
    n_steps=5,
    gamma=0.99,
    gae_lambda=1.0,
    ent_coef=0.0,
    vf_coef=0.5,
    max_grad_norm=0.5,
    learning_rate=7e-4,
    use_rms_prop=True,
    verbose=1,
    seed=42
)
```



PARÁMETROS EN SB3

- verbose=1: Nivel de verbosidad. 1 generalmente significa que imprimirá información de progreso durante el entrenamiento. 0 sería silencioso, 2 podría dar más detalles.
- seed=42: Semilla aleatoria. Fija la semilla para las operaciones aleatorias (inicialización de pesos, muestreo de acciones, aleatoriedad del entorno si la soporta) para que los resultados del entrenamiento sean reproducibles.

```python
# creo el agente A2C
model = A2C(
    policy='MlpPolicy',
    env=train_env,
    n_steps=5,
    gamma=0.99,
    gae_lambda=1.0,
    ent_coef=0.0,
    vf_coef=0.5,
    max_grad_norm=0.5,
    learning_rate=7e-4,
    use_rms_prop=True,
    verbose=1,
    seed=42
)
```



REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- https://www.kaggle.com/discussions/getting-started/162377
- Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., & Freitas, N. (2016, June). Dueling network architectures for deep reinforcement learning. In International conference on machine learning (pp. 1995-2003). PMLR.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., ... & Kavukcuoglu, K. (2016, June). Asynchronous methods for deep reinforcement learning. In International conference on machine learning (pp. 1928-1937). PmLR.
- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018, July). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In International conference on machine learning (pp. 1861-1870). Pmlr.
- Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., ... & Levine, S. (2019). Soft actor-critic algorithms and applications. arXiv preprint arXiv:1812.05905.
- https://github.com/ray-project/ray/tree/master/rllib/tuned_examples/sac